

Lab 1.5 - Quick Streamlit Tutorial

Streamlit in 15 minutes

From the [Documentation](#): [Streamlit](#) is an open-source Python framework for data scientists and AI/ML engineers to deliver dynamic data apps with only a few lines of code.

Key Features to know:

- Turns a `.py` script into a shareable web app by typing `streamlit run app.py`
- Zero front-end code required - so you can focus on data & ML

The purpose of this Lab is to not teach you Streamlit completely, but to give you a quick overview of the tool so you can immediately deploy your first model. You'll learn how to build and run a simple interactive application that uses a pre-trained machine learning model.

Streamlit is great for quick prototypes; however, we are going to learn more scalable tools for developing ML apps in the course ahead. You are welcome to learn more Streamlit from the links in the 'Additional Resources' section at the bottom of this page.

1. Installation:

I would highly recommend using a virtual environment to develop with Streamlit. If you're not familiar with setting up a virtual environment, you can follow the instructions on this page: <https://docs.streamlit.io/get-started/installation/command-line>

Once you activate your virtual environment, simply use pip to install streamlit on your terminal:

```
pip install streamlit
```

then, test the installation with:

```
streamlit hello or python -m streamlit hello
```

 (whichever works)

2. Run a simple 'Hello World' app

Let's start with the simplest possible app.

1. Create a new Python file named `hello_app.py`.
2. Add the following code:

```
import streamlit as st
st.title('Hello World')
st.write("This is my first Streamlit app!")
```

3. Now, run the app. Go to your terminal, navigate to the directory where you saved the file, and execute:

```
streamlit run hello_app.py
```

Your web browser will automatically open with your new app. You should see a title and a line of text.

To close the app on the terminal, and free the port, press `ctrl+c`

3. Essential Display and Input Widgets in Streamlit

Streamlit's magic comes from its widgets, which are simply function calls that add interactive elements to your app. For this, create a simple app.py script and enter the code below. This code contains comments to assist you with getting a quick idea on what the widgets do. I highly recommend trying along on your computer if you still aren't.

```
import streamlit as st
import pandas as pd

# --- TEXT ELEMENTS ---
st.title('Streamlit Basics Tutorial')
st.header('This is a header')
st.subheader('And this is a subheader')
st.markdown("""
This is Markdown. You can use it for:
* Lists
* **Bold** or *italic* text
* And more!
""")

# --- INTERACTIVE WIDGETS ---
st.header('Interactive Widgets')

# Slider
age = st.slider('How old are you?', min_value=18, max_value=100, value=25)
st.write(f'You selected: {age} years old.')

# Button
if st.button('Say Hello'):
    st.write('Hello there!')
else:
    st.write('Goodbye')

# --- DISPLAYING DATA ---
st.header('Displaying Data with Pandas')

df = pd.DataFrame({
    'first column': [1, 2, 3, 4],
    'second column': [10, 20, 30, 40]
})

st.write("Here is our sample DataFrame:")
st.dataframe(df)
```

Save the file. Your browser tab with the app should automatically update. If not, you can select "Rerun" in the top-right corner. Play with the slider, selectbox, and button to see how the app instantly responds. **This is the core concept: every time you interact with a widget, the script reruns.**

4. Creating an ML model to deploy

Before we head onto Streamlit, let's train and save a very simple model that we can later load in our Streamlit app for inference.

```
from sklearn.datasets import load_iris
from sklearn.ensemble import RandomForestClassifier
import joblib

iris = load_iris()
X, y = iris.data, iris.target

model = RandomForestClassifier(n_estimators=100, random_state=42)
model.fit(X, y)

# Save the model to a file
joblib.dump(model, 'model.pkl')
joblib.dump(iris.target_names, 'iris_target_names.pkl')

print("Model trained and saved!")
```

Run the python script above once to create `model.pkl`.

5. Let's Build a ML app

Write the script below an app.py file. Comments are included to give you an idea of the app structure.

```
import streamlit as st
import pandas as pd
import joblib

# --- Caching the model ---
# Use st.cache_data to load the model only once
@st.cache_data
def load_model():
    """Loads the pre-trained model and target names."""
    model = joblib.load('model.pkl')
    target_names = joblib.load('iris_target_names.pkl')
    return model, target_names

# Load the model and class names
model, target_names = load_model()

# --- App Layout ---

# 1. Title
st.title('Iris Flower Species Prediction')

# 2. Sidebar for User Input
st.sidebar.header('Input Features')

def user_input_features():
    """Creates sliders in the sidebar for user input."""
    sepal_length = st.sidebar.slider('Sepal length (cm)', 4.3, 7.9, 5.4)
    sepal_width = st.sidebar.slider('Sepal width (cm)', 2.0, 4.4, 3.4)
    petal_length = st.sidebar.slider('Petal length (cm)', 1.0, 6.9, 1.3)
    petal_width = st.sidebar.slider('Petal width (cm)', 0.1, 2.5, 0.2)

    data = {'sepal_length': sepal_length,
            'sepal_width': sepal_width,
            'petal_length': petal_length,
            'petal_width': petal_width}

    features = pd.DataFrame(data, index=[0])
    return features

user_input = user_input_features()

# Display the user input
st.subheader('Your Input:')
st.dataframe(user_input)

# 3. Prediction and Results
st.subheader('Prediction:')
prediction = model.predict(user_input)
prediction_proba = model.predict_proba(user_input)

# Display the predicted species
st.write(f'The predicted flower species is **{target_names[prediction][0].capitalize()}**.')

# Display the prediction probabilities
st.subheader('Prediction Probabilities:')
st.bar_chart(pd.DataFrame(prediction_proba, columns=target_names))
```

Now you can run this app with: `streamlit run app.py`

`@st.cache_data` : A "decorator" that tells Streamlit to cache the output of a function. This is crucial for performance. We use it here to ensure our (potentially large) ML model is only loaded from disk once, not every time a slider is moved.

You now have an interactive web app!

- The **sidebar** contains four sliders for the Iris flower features.
- The **main page** shows the values you selected, the model's prediction, and a bar chart of the prediction probabilities.
- Move the sliders and watch the prediction and probabilities update in real-time.

That's it! That's primarily all you need to know to build an ML app with streamlit for live inference. This should be enough to complete your first HW assignment.

You have successfully built and deployed a machine learning model as an interactive web application using Streamlit. You learned how to:

- Install and run Streamlit.
- Use basic text and interactive widgets.
- Display data using Pandas DataFrames and charts.
- Structure an app with a sidebar.
- Load and cache a pre-trained model for efficient predictions.

6. Additional Resources:

If got time and you're interested in learning more about Streamlit concepts, I'll highly recommend looking into the following pages from the official Documentation:

- Basic concepts: <https://docs.streamlit.io/get-started/fundamentals/main-concepts>
- Advanced concepts: <https://docs.streamlit.io/get-started/fundamentals/advanced-concepts> (covers caching and session state - interesting topics to learn)
- Create an app: <https://docs.streamlit.io/get-started/tutorials/create-an-app>